

Ethereum Protocol Fellowship, Cohort 7

EIP-8435

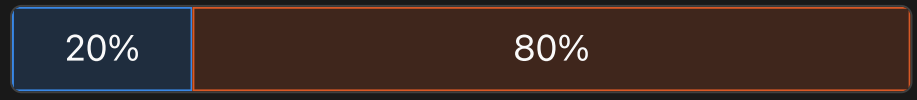
Write-age metadata for the Partitioned Binary Tree, with `last_written_block` in every leaf

Final presentation by **Astrion**, mentored by **Wei Han Ng**

Motivation

80%

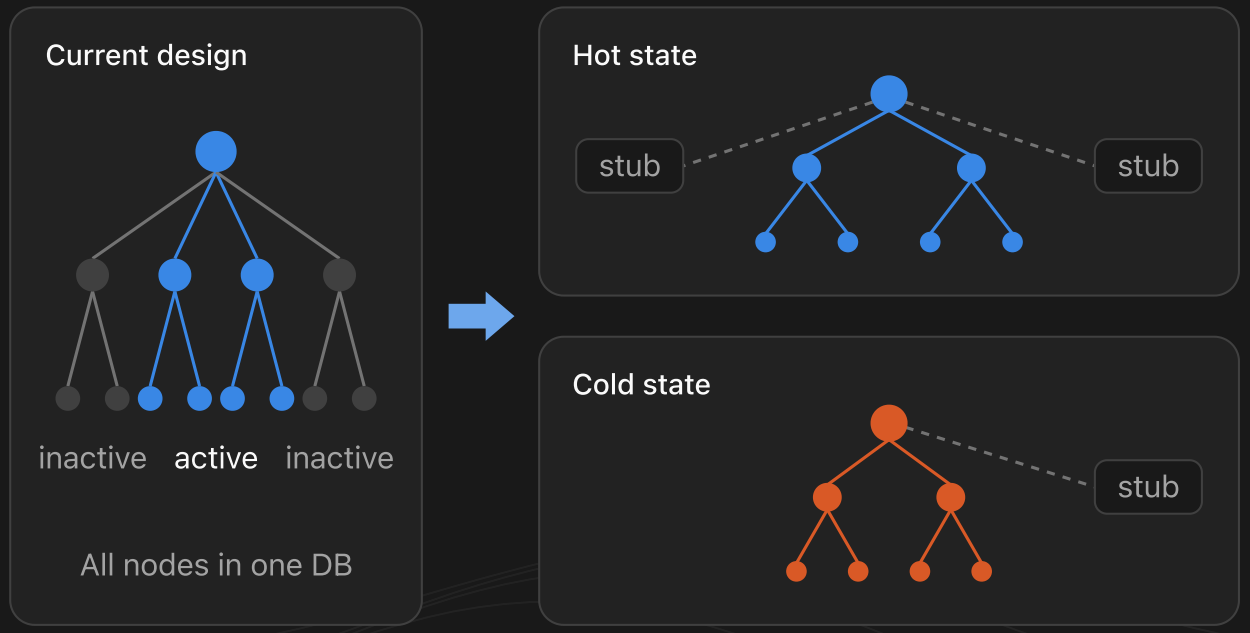
of Ethereum state has not been accessed in over a year



- Hot, used in the last year
- Cold, idle for over a year

Sources: EIP-8188 ACD slides, and “Not All State is Equal” on Ethereum Magicians.

Where it started



The EPF plan

State tiering in Reth with EIP-8188 and EIP-8295

5

`reth db` commands on my fork, to backfill write age, find inactive subtrees and move them to a cold file

github.com/astrion-coder/reth

Why the pivot

Hegotá

EIP-8188 was declined for inclusion

Declined for inclusion means the MPT version isn't shipping in that fork.

→

EIP-8297

The switch to the PBT is expected

A new tree is coming, and every client will rebuild its state for it.

→

Wei Han's suggestion

Put the tiering metadata into the PBT

Design it into the new tree from the start, which became EIP-8435.

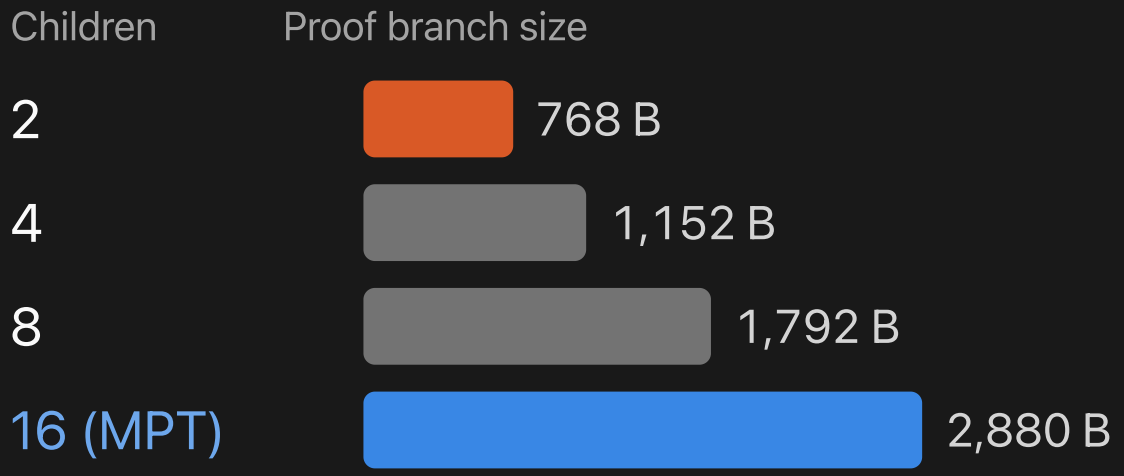
Source: EIP-8081, Hardfork Meta: Hegotá (EIP-8188 listed as Declined for Inclusion)

From MPT to PBT

EIP-8297 aims for blocks that can be proved, with small witnesses.

	MPT today	PBT in EIP-8297
Children	16 per node	2 per node
Encoding	RLP	Fixed byte offsets
Shape	A trie of tries	One tree for accounts, code and storage
Code	Outside the tree	Chunked in, shared by code hash
Hash	Keccak	Not final, BLAKE3 for now

Why binary

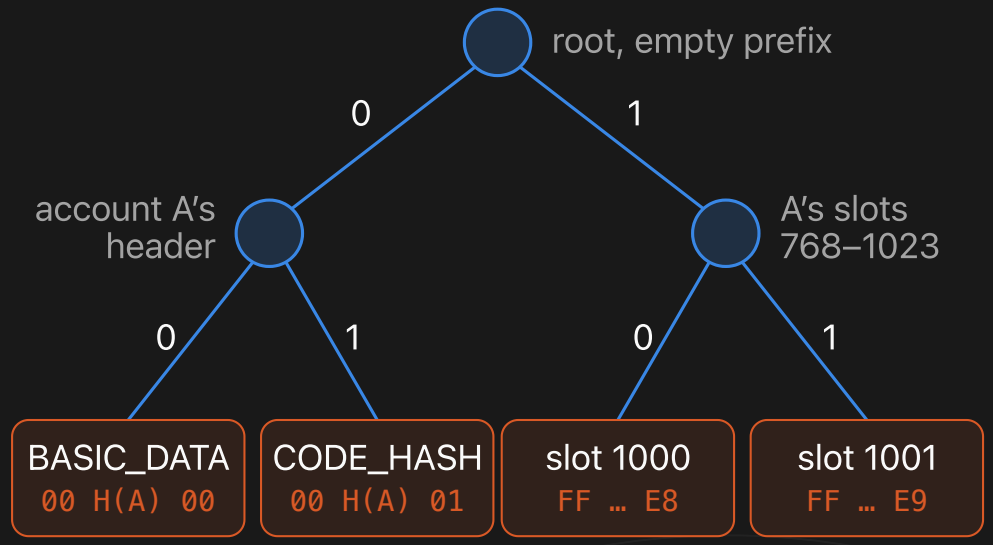


3.75×

smaller branch than with 16 children

Average branch for 2^{24} (about 16.8 million) leaves, from the rationale of EIP-8297.

Two node types



BranchNode

The prefix (the bits every key below shares), left and right

```
H(0x01 || prefix || L || R)
```

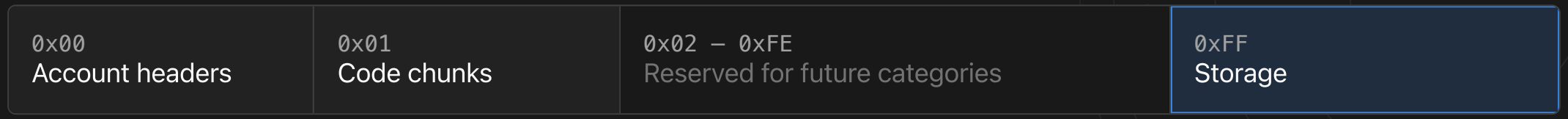
LeafNode

The complete key and a 32-byte value

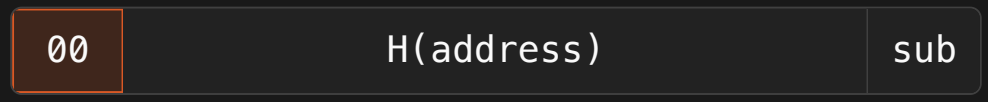
```
H(0x00 || key || value)
```

There are no extension nodes, because the prefix keeps the tree shallow.

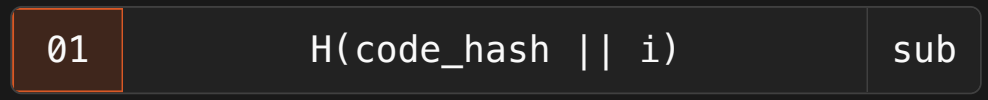
Keys and zones



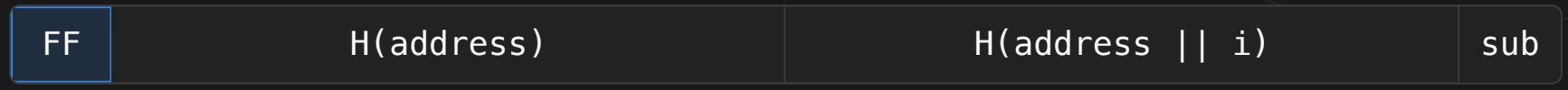
Account
34 bytes



Code
34 bytes



Storage
66 bytes



`i` picks a group of 256 slots or code chunks, and `sub` picks one inside it. Keys that differ only in `sub` share a stem, so they sit together.

The account header

One stem per account, `00 || H(address) || sub`, with 256 sub-indices.

The header holds the account's `BASIC_DATA`, its `CODE_HASH` and its storage slots 0–63, all under that one stem.

BASIC_DATA

Version, code_size, nonce and balance at fixed offsets in one 32-byte value. Bytes 1–3 are reserved.

Slots 0–63

Live next to the account's header. Slot 64 and up go to the storage zone.

Three rules that matter

01	Every value is 32 bytes	There is no RLP list to append a field to
02	Writing zero deletes	Zero means absent, so a cleared slot leaves no leaf
03	No storage root	Account and slot leaves are written independently

> So where does `last_written_block` go? EIP-8188 defines it only for the MPT, as an extra RLP field.

Final deliverable

EIP-8435

Last-Written Block in PBT Leaves

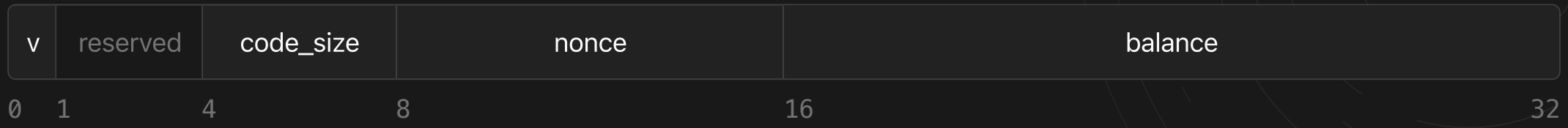
Packs `last_written_block` write-age metadata into the account and storage leaves of the Partitioned Binary Tree.

Standards Track | Core | Draft, merged

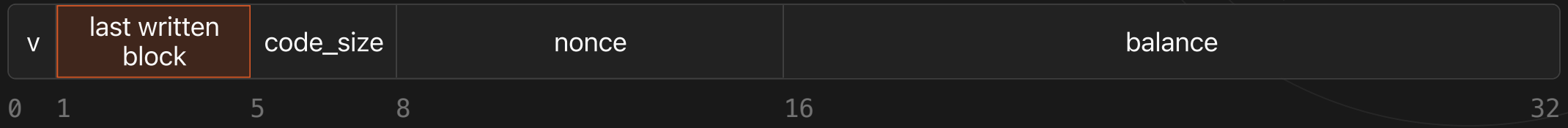
eips.ethereum.org/EIPS/eip-8435

+0 bytes in the account leaf

EIP-8297 `BASIC_DATA`, 32 bytes



EIP-8435 `BASIC_DATA`, still 32 bytes



`code_size`	Shrinks from 4 bytes to 3, which still holds 16,777,215, about 256× the 64 KiB limit of EIP-7954
nonce and balance	Same offsets, same widths

+0 bytes

per account, where EIP-8188 adds 5

Why 3 bytes is enough

256× headroom over the code size limit



EIP-7864, the earlier unified binary tree

v	reserved	code_size	nonce	balance

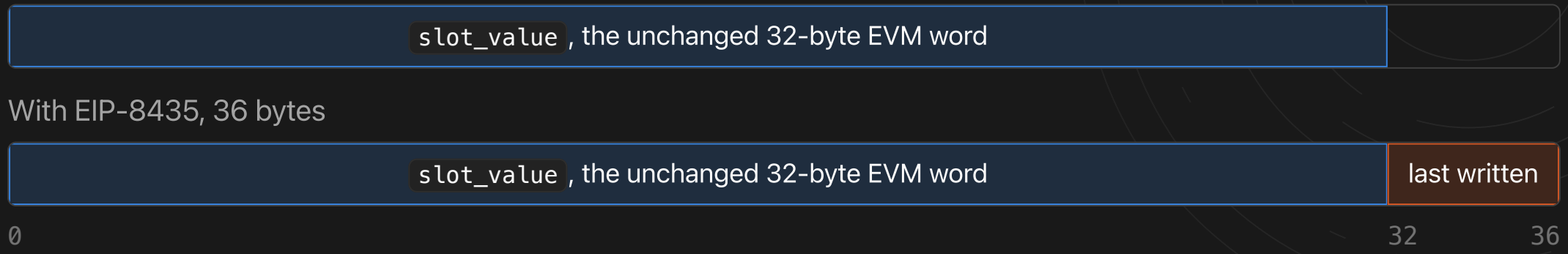
EIP-8435

v	last written	code_size	nonce	balance

Both put `code_size` at offset 5, as 3 bytes. From EIP-7864 and EIP-7954.

+4 bytes in the storage leaf

Storage leaf value in EIP-8297, for slots 0–63 and the storage zone alike



Extra state across ~1.5 billion slots



Hashing cost is unchanged

The BLAKE3 input stays within 2 blocks, at up to 103 bytes.

State-creation gas becomes `STATE_BYTES_PER_STORAGE_SET + 4` (EIP-8037).

When it changes

Writes set it

- Balance transfers
- Nonce bumps, including EIP-7702
- Fee-recipient credits
- Withdrawals (EIP-4895)
- New accounts and slots
- Slot value changes

Reads never do

- `SLOAD` and `BALANCE`
- `EXTCODESIZE`, `EXTCODECOPY` and `EXTCODEHASH`
- Reading the field itself

Reverts restore it

Journaled and rolled back with its own leaf

> **No cascade.** A storage write never touches the account leaf.

Retrieval at will

The block number travels inside the leaf

Wherever the leaf goes, its write age goes with it. State that existed before activation reads 0 until its first write.

Read-only

No execution, no side index, no state change

Anywhere

Hot DB, cold blob, snapshot or block witness

Verifiable

The leaf hash covers it, so an inclusion proof proves it

From draft to merged

Oct 4, 2026 — **Drafted**
Last-Written Block in PBT Leaves

Oct 4, 2026 — **PR #12420**
Opened on ethereum/EIPs

Oct 4 to 24, 2026 — **CI and review**
Gaps found and fixed

Oct 24, 2026 — **Merged**
With status Draft

20 days

from the first draft to the merge

Where EIP-8435 stands

Draft	Review	Last Call	Final

The statuses an EIP moves through, from EIP-1

Discussion on the Ethereum Magicians thread and in PR #12420.

Outcomes

Standard

EIP-8435

Merged as Draft, co-authored with Wei Han Ng

Exploration

Reth prototype

EIP-8188 tiering on Reth, which led to the EIP

Writing

Dev updates

Weekly notes from week 0 to 16, plus the Magicians thread

The prototype shaped the standard, because a cold store needs the write age inside the leaf.

Lessons and next steps

01 Feedback on Eth Magicians

02 Testing and Benchmarks

03 A client prototype (or glam-devnet-8)

Acknowledgements

People

Wei Han Ng

First Author of 8188 and reviewed early drafts of 8435

Mario Havel

Organiser of EPF

Organisations

Ethereum Foundation

Hosted EPF



Thank you

Questions, comments and ideas are welcome.

I'm @astrion0587 on Discord.

eips.ethereum.org/EIPS/eip-8435

Ethereum Magicians thread